

MyVENSIM — Refatoração da Sprint 5

Arquitetura em Camadas: Componentização e Fábricas (Model como Fábrica)

David Souza Nascimento — BCC322 Engenharia de Software I (UFOP)

11 de junho de 2026

Sumário

1. Contexto	1
2. Resumo da nova arquitetura	2
3. Mudanças por componente	2
3.1. System / SystemImpl	2
3.2. Flow / FlowImpl	2
3.3. Model / ModelImpl	4
4. Build (Makefile)	5
5. Testes atualizados	5
5.1. Testes unitários (test/unit)	5
5.2. Testes funcionais (test/funcional)	5
5.3. Resultado da execução	5
6. Documentação Doxygen	6
7. Pendências e observações finais	6
Sugestão de commit	6
8. Conclusão	7

1. Contexto

Este relatório documenta a refatoração realizada no projeto **MyVENSIM** (simulador de dinâmica de sistemas em C++), referente à Sprint 5 — “*Arquitetura em Camadas: Componentização + Fábricas*”.

A refatoração foi motivada pela revisão do material da **Aula 11 — Design Patterns**, que apresentou um novo formato de API para a fábrica de **Systems** e **Flows**, substituindo a abordagem anterior (uma classe **Factory** separada) por um modelo em que a própria classe **Model** atua como fábrica e dona dos objetos que cria.

O objetivo deste documento é registrar, de forma organizada, o que mudou no código, por quê, e o resultado da validação (compilação e testes).

2. Resumo da nova arquitetura

Na versão anterior (Sprint 5 inicial), `System` e `Flow` eram criados por uma classe `Factory` independente, e `Flow` expunha diretamente os métodos `setSource`/`setTarget`/`clearSource`/`clearTarget`.

Na nova versão, `Model` passa a concentrar três responsabilidades:

1. **Fábrica** — cria `Systems` (`createSystem`) e `Flows` (`createFlux<T_FLUX_IMPL>`, um *template method*);
2. **Dono dos objetos** — guarda ponteiros para todos os `Systems` e `Flows` criados e é responsável por destruí-los;
3. **Gerenciador de conexões** — `setSource`, `setTarget`, `clearSource` e `clearTarget` passam a ser operações do `Model` sobre um `Flow`, em vez de métodos públicos do próprio `Flow`.

A figura abaixo resume as classes/interfaces principais e seus relacionamentos após a refatoração:

Pontos-chave do diagrama:

- `System` e `Flow` continuam sendo **interfaces puras** (sem atributos, apenas métodos virtuais puros), implementadas por `SystemImpl` e `FlowImpl`.
- `Model` também é uma interface, implementada por `ModelImpl`.
- `ModelImpl` **cria e possui** (relação de composição/diamante cheio) todos os `SystemImpl` e `FlowImpl` que instancia — quando um `ModelImpl` é destruído, ele libera toda essa memória.
- `FlowImpl` mantém ponteiros (`source/target`) para `System`, manipulados apenas através do `Model`.

3. Mudanças por componente

3.1. `System` / `SystemImpl`

- A interface `System` ganhou os métodos `getName()` e `setName(...)`, além dos já existentes `getValue()/setValue(...)`.
- `SystemImpl` passou a ter um atributo público `id_` (string), usado como identificador/nome do sistema.
- O construtor agora é `SystemImpl(string id, double value = 0.0)`.
- Construtor de cópia e operador de atribuição continuam **privados** (objetos `SystemImpl` não devem ser copiados — são geridos pelo `Model`).

3.2. `Flow` / `FlowImpl`

- A interface `Flow` foi **reduzida ao mínimo**: `getSource()`, `getTarget()` e `execute()`. Os métodos de conexão saíram da interface.
- `FlowImpl` ganhou um atributo `id_` (string) e o construtor passou a ter a assinatura `FlowImpl(string id = "", System* source = nullptr, System* target = nullptr)` — exatamente os 3 parâmetros exigidos pelo `Model::createFlux<T>`.
- `setSource`, `setTarget`, `clearSource` e `clearTarget` continuam implementados em `FlowImpl`, porém **não fazem mais parte da interface `Flow`** — são métodos concretos chamados pelo

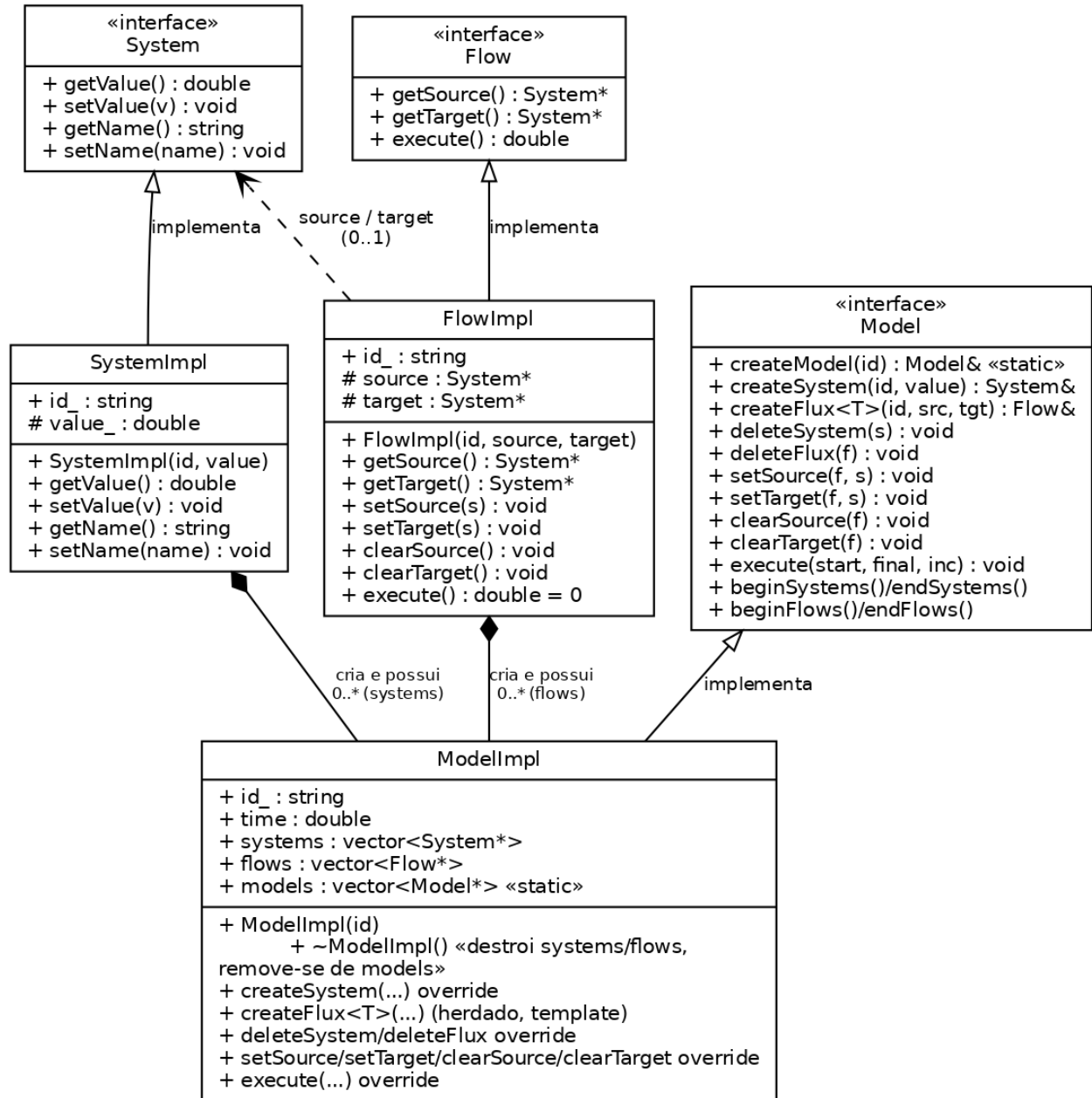


Figure 1: Diagrama de classes da nova arquitetura

Model através de `static_cast<FlowImpl&>`.

- `execute()` permanece puro virtual: cada fluxo concreto (`ExponentialFlow`, `LogisticFlow`, `ComplexFlow`, etc.) define sua própria equação.

3.3. Model / ModelImpl

Esta foi a maior mudança da refatoração.

Criação (fábrica):

```
Model& m = Model::createModel("meuModelo");
```

```
System& s1 = m.createSystem("ovos", 10.0);
```

```
System& s2 = m.createSystem("galinhas", 0.0);
```

```
Flow& f = m.createFlux<FluxoExp>("nascimento", &s1, &s2);
```

- `Model::createModel(string id)` é um método **estático** que cria e devolve uma `Model&` (implementado em `ModelImpl`, registrado no vetor estático `models`).
- `createSystem` cria um `SystemImpl`, adiciona-o à coleção interna (`vector<System*> systems`) e devolve uma referência.
- `createFlux<T_FLUX_IMPL>` é um **método template** (definido inline em `model.hpp`, pois templates precisam estar visíveis no ponto de uso): instancia `T_FLUX_IMPL(id, source, target)`, adiciona à coleção `flows` e devolve uma referência.

Posse e ciclo de vida:

- `ModelImpl` guarda `vector<System*> systems` e `vector<Flow*> flows` e é a **única dona** desses objetos.
- O destrutor de `ModelImpl` percorre as duas coleções, faz `delete` em cada ponteiro e limpa os vetores — quem usa o `Model` não precisa (e não deve) deletar `Systems/Flows` manualmente.
- `deleteSystem(System&)` e `deleteFlux(Flow&)` permitem remoção individual antes da destruição do `Model`.

Registro estático de Models:

- `ModelImpl` mantém `static vector<Model*> models` com todos os `Models` vivos.
- Cada `ModelImpl` se registra nesse vetor ao ser construído.
- No destrutor, o `ModelImpl` se **remove** do vetor `models`, evitando que sobre um ponteiro pendurado (*dangling pointer*) caso o vetor seja consultado depois.

Conexões entre Systems e Flows:

- `setSource(Flow&, System&)`, `setTarget(Flow&, System&)`, `clearSource(Flow&)` e `clearTarget(Flow&)` agora são métodos do `Model`. Internamente, fazem `static_cast<FlowImpl&>(f)` para acessar os métodos concretos de conexão de `FlowImpl`.

Simulação:

- `execute(start, final_time, inc)` mantém o mesmo algoritmo de antes: a cada passo, calcula `execute()` de todos os `Flows`, depois aplica o resultado (subtrai do `source`, soma ao `target`) e avança o tempo.

4. Build (Makefile)

- A classe `Factory` (`factory.hpp` / `factory.cpp`) **saiu do build**: removida da lista `SRC` da regra `lib`.
- A biblioteca `bin/libmyvensim.so` agora é compilada apenas a partir de `systemImpl.cpp`, `flowImpl.cpp` e `modelImpl.cpp`.
- Os arquivos `src/factory.hpp` e `src/factory.cpp` **não puderam ser apagados fisicamente** (bloqueio do OneDrive ao deletar arquivos sincronizados). Eles continuam na pasta `src/`, porém **órfãos** (não são mais referenciados por nenhum teste ou pelo Makefile). Podem ser apagados manualmente quando for conveniente.

5. Testes atualizados

5.1. Testes unitários (`test/unit`)

- `unit_System`: reescrito para instanciar `SystemImpl` diretamente (`SystemImpl s1("s1", 100.0)`), incluindo novos casos para `getName()/setName()`.
- `unit_Flow`: a classe de teste `ConcreteFlow` passou a herdar de `FlowImpl` com construtor de 3 parâmetros (`ConcreteFlow(id, source, target) : FlowImpl(id, source, target) {}`). `Systems` de teste agora são objetos de pilha (`SystemImpl s1(...)`), e seus endereços (`&s1`) são passados ao `Flow`.
- `unit_Model`: conjunto de testes totalmente novo, cobrindo: `createModel` (construtor/destrutor), `createSystem`, `createFlux<T>`, `deleteSystem`, `deleteFlux`, `setSource`, `setTarget`, `clearSource`, `clearTarget`, `beginSystems/endSystems`, `beginFlows/endFlows` e `execute` (com o cenário exponencial clássico).

5.2. Testes funcionais (`test/funcional`)

- `funcional_tests.cpp` foi reescrito usando a nova API (`Model::createModel`, `model.createSystem`, `model.createFlux<T>`), mantendo os três cenários originais:
 - **Exponencial** (`ExponentialFlow`)
 - **Logístico** (`LogisticFlow`)
 - **Complexo** (`ComplexFlow`, com 5 sistemas e 6 fluxos)

5.3. Resultado da execução

```
=====
MyVensim - Testes Unitarios - Sprint 4
=====
[OK] Globals
[OK] System
[OK] Flow
[OK] Model
=====
Todos os testes passaram com sucesso!
=====

Iniciando testes funcionais...
[OK] -> Teste Exponencial passou com sucesso!
```

[OK] -> Teste Logístico passou com sucesso!

[OK] -> Teste Complexo passou com sucesso!

Os valores numéricos finais dos testes funcionais permanecem **idênticos** aos da versão anterior (validação de que a refatoração não alterou o comportamento da simulação):

Cenário	Resultados (valores brutos, truncados)
Exponencial	pop1 = 366032, pop2 = 633968
Logístico	p1 = 882167, p2 = 217833
Complexo	q1=318513, q2=184003, q3=771143, q4=561728, q5=164612

6. Documentação Doxygen

- Os comentários Doxygen (`@brief`, `///`, `/** ... */`) foram revisados e completados em todos os headers refatorados (`system.hpp`, `flow.hpp`, `flowImpl.hpp`, `model.hpp`, `modelImpl.hpp`, `systemImpl.hpp`), cobrindo a nova API (`createModel`, `createSystem`, `createFlux<T>`, `deleteSystem`, `deleteFlux`, `setSource/Target`, `clearSource/Target`, iteradores, etc.).
- Importante:** as pastas `sprint02/html/` e `sprint02/latex/` (saída gerada pelo Doxygen) ainda refletem a versão **anterior** do código (com a classe `Factory`). Não foi possível regenerá-las neste ambiente porque o executável `doxygen` não está disponível no sandbox usado para a refatoração.
- Para atualizar a documentação gerada, basta rodar, na pasta `sprint02/`:

```
doxygen Doxyfile
```


(o `Doxywizard/doxygen` já estão instalados no seu computador).

7. Pendências e observações finais

- `src/factory.hpp` / `src/factory.cpp` — arquivos órfãos, não compilados nem usados. Podem ser apagados manualmente pelo Explorador de Arquivos.
- Documentação gerada (html/latex)** — está desatualizada em relação ao novo código; rodar `doxygen Doxyfile` para regenerar.
- Commit/push** — as alterações da refatoração estão prontas no diretório de trabalho (`sprint02/src`, `sprint02/test`, `sprint02/Makefile`), mas não foi possível executar `git add/git commit` neste ambiente: o arquivo `.git/index.lock` ficou travado pelo OneDrive e não pôde ser removido. Sugestão de mensagem de commit e lista de arquivos estão no fim deste documento.
- Capturas de tela (“prints”)** — não foi possível localizar arquivos de imagem das capturas mencionadas; portanto este relatório usa um diagrama de classes gerado a partir do código (Graphviz) em vez delas.

Sugestão de commit

```
git add sprint02/Makefile \  
    sprint02/src/system.hpp sprint02/src/systemImpl.hpp sprint02/src/systemImpl.cpp \  
    sprint02/src/flow.hpp sprint02/src/flowImpl.hpp sprint02/src/flowImpl.cpp \  
    sprint02/test/
```

```
sprint02/src/model.hpp sprint02/src/modelImpl.hpp sprint02/src/modelImpl.cpp \
sprint02/test/unit/unit_System.h sprint02/test/unit/unit_System.cpp \
sprint02/test/unit/unit_Flow.h sprint02/test/unit/unit_Flow.cpp \
sprint02/test/unit/unit_Model.h sprint02/test/unit/unit_Model.cpp \
sprint02/test/funcional/funcional_tests.cpp
```

```
git commit -m "refactor(sprint5): Model como fabrica e dono de Systems/Flows"
```

- Model::createModel estatico cria instancias de ModelImpl
- createSystem/createFlux<T> criam e registram Systems/Flows no Model
- Model passa a possuir e destruir Systems/Flows (destrutor + deleteSystem/deleteFlux)
- registry estatico ModelImpl::models, com auto-remocao no destrutor
- setSource/setTarget/clearSource/clearTarget movidos de Flow para Model
- System e Flow ganham id_ (System ganha getName/setName)
- Flow::execute simplificado para 3 metodos na interface
- factory.hpp/.cpp removidos do Makefile (arquivos ainda presentes, nao usados)
- testes unitarios e funcionais reescritos para a nova API; todos passando"

Antes de rodar esses comandos, remova manualmente o arquivo .git/index.lock (na raiz do repositório `cpp-project-template`), caso o Git acuse “Another git process seems to be running”. O `git push` deve ser feito por você, conforme combinado.

8. Conclusão

A refatoração migrou com sucesso o MyVENSIM do modelo de **fábrica externa (Factory)** para o modelo de **Model como fábrica e dono dos objetos**, conforme apresentado na Aula 11. Todas as interfaces (System, Flow, Model) foram simplificadas, a posse de memória ficou centralizada e explícita, e o registro estático de Models evita ponteiros pendurados. A suíte de testes (unitários e funcionais) foi totalmente adaptada à nova API e **100% dos testes continuam passando**, com os mesmos resultados numéricos de antes — confirmando que a mudança foi estrutural, sem alterar o comportamento